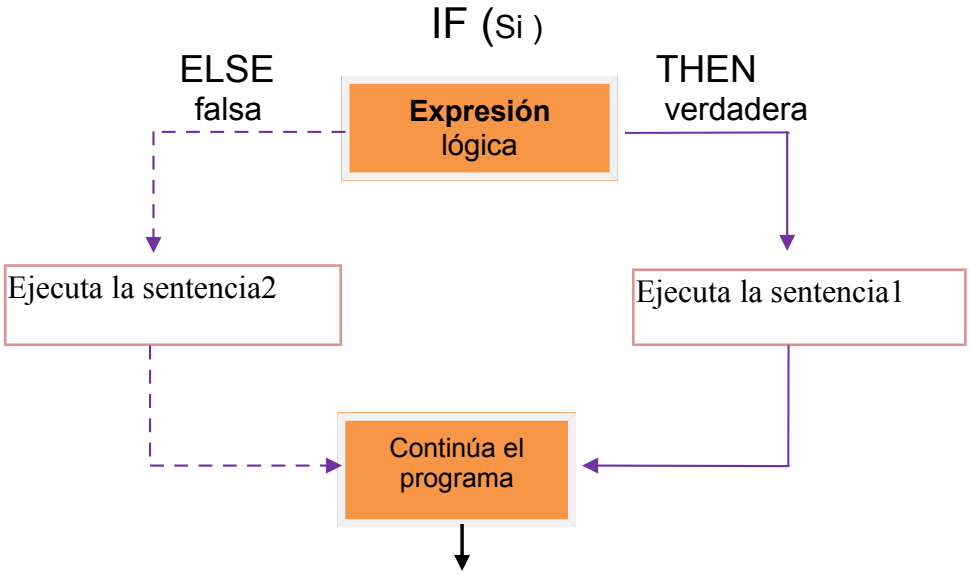
Capítulo 3

Estructuras control de ejecución: Decisión

1. Estructura de control de ejecución: decisión IF-THEN-ELSE

Frecuentemente es necesario especificar dos o más formas de acción, y permitir que el proceso que sigue el programa seleccione una de ellas durante la ejecución. Una sentencia condicional (sentencias IF) selecciona una acción para ser ejecutada según el resultado de una expresión lógica.
La sentencia IF especifica que **sentencia1** (que sigue a THEN) debe ejecutarse sólo si una cierta condición (expresión lógica o booleana) es verdadera. Si es falsa, entonces se ejecuta **sentencia2**, que sigue a la palabra reservada ELSE:
Flujo de ejecución:



A continuación vemos tres casos de sintaxis Pascal para la sentencia IF.
1) Si la expresión lógica es verdadera (THEN) se ejecuta la sentencia1 y si es falsa (ELSE) se ejecuta la sentencia2:

IF expresión lógica **THEN**

Sentencia$_1$
ELSE
Sentencia$_2$;

2) En este caso sólo se pone de manifiesto la sentencia que se ejecuta si la condición es verdadera (no interesa el otro caso):

IF expresión lógica **THEN**

Sentencia;

3) En este caso recurrimos a enunciados compuestos si deseamos ejecutar varias sentencias (simples o compuestas) cuando la expresión lógica es verdadera (y eventualmente se puede indicar otro enunciado compuesto si esta es falsa):

IF expresión lógica **THEN**
  begin
    SentenciaV$_1$;
    SentenciaV$_2$;
    .....;
    SentenciaV$_n$;
  end
ELSE
  begin
    SentenciaF$_1$;
    SentenciaF$_2$;
    .....;
    SentenciaF$_n$;
  end;

E1) Se lee un par de coordenadas (x,y) de un punto del plano. Verificar si se encuentra en el círculo centrado en el origen y con radio 5.

```
Program deciE1;
VAR
   x,y:real;
BEGIN
   write('Ingrese dos valores reales');
   readln(x,y);
   IF x*x+y*y<=25 THEN
       writeln('punto del círculo')
   ELSE
       writeln('punto fuera del circulo')
END.
```

Observación: recordar que el símbolo punto y coma (;) separa sentencias. NO debe escribirse antes de la cláusula ELSE y es de uso optativo luego antes del delimitador END porque a continuación no hay otra sentencia.

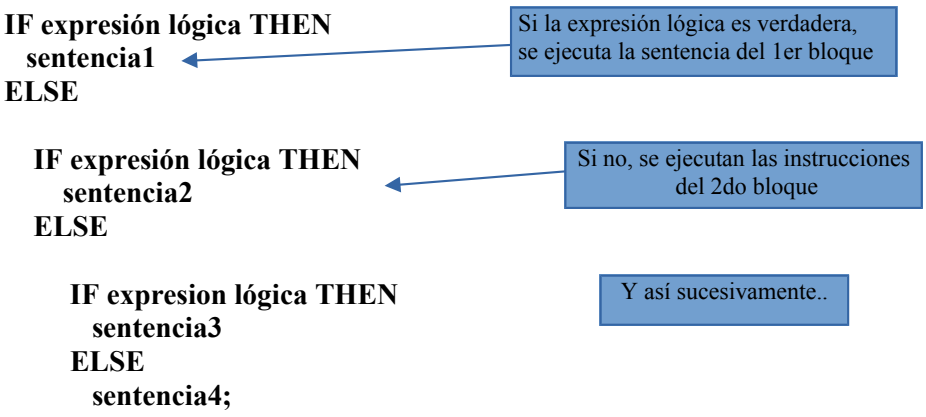
2. Sentencias de decisión anidadas

Se denomina *anidamiento* al uso de una estructura dentro de otra, es decir, un bloque de código que pertenece a una estructura contiene otros bloques de código dentro y dentro de éstos últimos pueden existir nuevos bloques[1]. Este concepto no solo se aplica a la Sentencia IF, sino que es posible anidar en todas las demás estructuras que se verán en los demás capítulos. . En este caso tendremos sentencias IF anidadas.

Un IF anidado es un IF que es la sentencia de otro IF o ELSE. Una cláusula ELSE siempre se refiere al IF más próximo que esté en el mismo bloque.

Es posible encadenar varios **IF**s y varios **ELSE**S en algo conocido como escalera **IF-ELSE-IF** por su apariencia visual.
Forma General:

1 Lage, F., Cataldi, Z. Y Salgueiro, F. (2008). Fundamentos de algortimos y progrmación. Ed. Nueva Librería. Buenos Aires. Argentina.



Las expresiones lógicas son evaluadas de arriba hacia abajo. Tan pronto se cumpla con una expresión lógica, las sentencias asociadas a ella son ejecutadas y el resto de la escalera es saltado. Si ninguna de las condiciones son ciertas, el último else es ejecutado (si está presente).

E2) Analizar si la suma de dos números cualesquiera es positiva o negativa. En caso de ser cero, indicar que son números opuestos.

```
Program anidamientoE4;
VAR
   a,b,suma:real;
BEGIN
   Readln(a,b);
   suma:=a+b;
   IF suma>0 THEN
      writeln('suma positiva')
   ELSE
      IF suma<0 THEN
           writeln('suma  negativa')
        ELSE
            writeln('numeros opuestos')
END.
```

Este ejemplo representa la anidación con estructuras **IF.** Si suma es mayor que 0, escribiremos “suma Positiva”. Si **suma** no es mayor que 0, entraremos en el siguiente nivel de anidación, que nos llevará al siguiente **IF**, comprobando si **suma** es menor que 0, si esto es cierto, escribiremos “suma Negativa” ; si por el contrario **suma** no es menor que 0, escribiremos “números Opuestos”.

En el programa `Anidamiento` hemos tratado 3 condiciones mutuamente excluyentes:

1. suma>0
2. suma<0
3. suma=0

Cada una de ellas desata acciones distintas. En estos casos el anidamiento de los IF IFs bien desarrollados, evita preguntas superfluas (por ejemplo, preguntar suma=0 está de más porque se controla con ELSE del segundo IF)

2.1 Comparación entre decisión secuencial y decisión anidada

Supongamos que el problema del ejemplo E4 lo desarrollamos con sentencias de decisión secuenciales. En ese caso tendremos la siguiente solución:

```
IF suma>0 THEN
      writeln('suma positiva');
IF suma<0 THEN
      writeln('suma  negativa');
IF suma=0 THEN
      writeln('numeros opuestos');
```

La secuencia de sentencias IF es ineficiente porque el procesador debe ejecutar todas las sentencias de decisión sin importar cuál es el resultado. Por ejemplo, si suma=-4 todas las sentencias IF evalúan cada expresión booleana como falsa, excepto la segunda que es verdadera y escribe **'suma Negativa'**
Independientemente del valor que tome la variable suma, se evaluarán **tres** expresiones booleanas :

1. suma>0
2. suma<0
3. suma=0

En el caso de la solución propuesta en E4, se evaluarán en el peor de los casos **dos** expresiones booleanas:

1. suma>0
2. suma<0

2.2 Indentación

A fin de evitar la ambigüedad sintáctica que surge de construcciones como la siguiente:

```
IF exp_booleana1 THEN IF exp_booleana2 THEN sentencia1
  ELSE sentencia2;
```

Conviene usar la ***indentación*** (sangrías que facilitan la lectura de las estructuras), y si desea, puede poner los paréntesis BEGIN/END a fin de incrementar la legibilidad y evitar la ambigüedad. La construcción equivalente es:

```
IF exp_booleana1 THEN
             BEGIN
                IF exp_booleana2 THEN
                    sentencia1
                ELSE
                    sentencia2
             END;
```

3. Algunas consideraciones

Podemos aumentar la **legibilidad** de un programa usando apropiadamente las expresiones booleanas.

E3) Podemos reemplazar los siguientes IF anidados

```
IF a<b THEN
        IF a<c THEN
              writeln(a);
```

por la siguientes sentencia IF donde se usa una expresión booleana más compleja:

```
IF (a<b) and (a<c) THEN
     writeln(a);
```

E4) Ingresar una letra minúscula, determinar si se trata de una vocal o consonante. Escribir carteles aclaratorios (uso de estructura: IF-THEN-ELSE con predicados compuestos:

```
Program deciE6;
Var
 car:char;
begin
 write('Ingrese una letra ');
 readln(car);
 IF (car='a') or (car='e') or (car='i') or (car='o') or (car='u') THEN
    writeln('La letra ingresada es una vocal')
 ELSE
    writeln('La letra ingresada es una consonante');
 end.
```

Observación:

Evitar el abuso de IF cuando se usa variables booleanas.
Ejemplo: Sea *encontrado* una variable booleana

IF a=b THEN
***encontrado*:= true**
ELSE
***encontrado*:=false;**

Más simple y en forma equivalente se tiene:

***encontrado*:= a=b;**

4. Sentencia CASE

Cuando en un programa se debe seleccionar entre una cantidad variable de acciones se aconseja el uso de la decisión generalizada (o decisión múltiple). CASE decide entre varias sentencias mutuamente excluyentes (todas con probabilidad de selección pareja). El uso de CASE presenta restricciones.

E7) Un cobrador de peaje tiene el siguiente cuadro de categorías y lo que debe cobrar:

Categoría	Vehículo	Precio
1 y 2	Bici y motos	Gratis
3	Autos	2.50
4	Autos con trailer	5
5	Camiones	10

```
Program peajeE7;
   TYPE
             Tipo=1..5;
   VAR
              cate:Tipo;
              tarifa:real;
```

```
            Error:Boolean;
   BEGIN
     Readln(cate);
     Error:=false;
     IF (cate=1)or(cate=2) THEN
        tarifa:=0
     ELSE IF cate=3 THEN
                tarifa:=2.5
          ELSE IF cate=4 THEN
                   tarifa:=5
                ELSE IF cate=5 THEN
                        tarifa:=10
                     ELSE
                        Error:=true;
      IF error THEN
         Write('nro. Equivocado')
      ELSE
         Writeln('La tarifa es =',tarifa:8:2)
    END.
```

El programa puede transformarse en el siguiente programa donde las sentencias IF anidadas son llevadas a la sentencia **CASE** equivalente:

```
Program peaje;

Type
      Tipo=1..5;
Var
      Cate:tipo;
      Tarifa:real;
      Error:Boolean;

Begin
   Readln(cate);
   Error:=false;
  CASE cate OF
        1,2 :tarifa:=0;
          3 :tarifa:=2.5;
          4 :tarifa:=5;
          5 :tarifa:=10;
    ELSE
          Error:=true
   END;
    IF error THEN
          Write('nro. Equivocado')
    ELSE
          Writeln('La tarifa es =',tarifa)

End.
```

La sintaxis de CASE es la siguiente**:**

CASE selector OF
Lista$_1$de constantes: Sentencia$_1$;
Lista$_2$ de constantes: Sentencia$_2$;
....;

Lista$_n$ de constantes: Sentencia$_n$;
[ELSE Sentencia$_{n+1}$;]
END;

- El tipo de dato de la expresión selector debe ser de tipo ordinal.
- CASE selecciona como sentencia a ejecutarse aquella cuya etiqueta (las constantes) es igual al valor actual del selector
- Si el valor actual de selector no encuentra un valor igual en alguna de las listas de constantes, ejecuta la sentencia que sigue a ELSE.
- ELSE es opcional (por tal motivo figura entre corchetes).
- Cada constante debe ser única dentro de cada CASE.

5. EJERCICIOS

Consignas de trabajo

- Para los enunciados que plantean problemas, desarrollar programas, claros yeficientes, en lenguaje Pascal.
- Otro tipo de actividad se especifica puntualmente
- Para los casos en que el programa Pascal indique uno o más Lotes de Prueba, realizar la prueba de escritorio para cada caso

1) A continuación se muestra la prueba de escritorio del programa EJEM1.

```
Program EJEM1;
var
    x:integer;
begin
    writeln('Ingrese un nro. entero: ');
    readln(x);
    IF x<2 THEN
       writeln('Primero')
    ELSE
       writeln('Segundo');
    IF x<4 THEN
       writeln('Tercero');
    writeln('Cuarto');
    readln;
end.
```

Lote de prueba: 3

X	X<2	X<4	SALIDA
3			
	F		
			Segundo
		V	
			Tercero
			Cuarto

1.1) Para qué valores de x la salida es:
Primero
Tercero
Cuarto

1.2) ¿Habrá algún valor de x para el cual se muestre sólo Primero? Justificar la respuesta.

2) Indicar qué valor se muestra para cada lote de prueba

```
Program prg2;
var
    a,b:integer;
begin
    writeln('Ingrese dos nros. Enteros: ');
    readln(a,b);
    IF a > b  THEN
       Begin
          writeln(Ln(a-b):5:1);
          If b<0 then writeln(b, ' ',abs(b));
       End
    Else
       Writeln(sqrt(b-a):5:1);
    readln;
end.
```

a) Lote de prueba : 3, 5
b) Lote de prueba : 4, -3
c) Lote de prueba : 4, 4

3) Evaluar la siguiente función definida por tramos para un valor de x.

$$f(x)=\begin{cases} 5x+2 \text{si } x\leq 3 \\ 4x-5 \text{si } x>3 \end{cases}$$

4) Determinar qué problemática resuelve el siguiente programa:

```
Program prg6;
var
    f,x:real;
begin
    readln(x);
    IF x<0  THEN
       f:= sqr(x)
    ELSE
       IF x<5 THEN
          f:= x+1
       ELSE
          IF x=5 THEN
             f:= sqrt(x)
          ELSE
             f:= x-1;
    writeln('El resultado es ', f:6:2);
    readln;
end.
```

5) Realizar la prueba de escritorio de los siguientes programas con los lotes de prueba dados y comparar los resultados:

5.1)

a) Lote de prueba : 6
b) Lote de prueba : 7
c) Lote de prueba : -4

```
Program condi5_1;
var
    y:integer;
begin
    writeln('escriba un número entero');
    readln(y);
    if y>0 then
       begin
          if not(odd(y)) then
              writeln(y div 2)
       end
    else
       writeln('El valor ingresado es ',y);
```

```
    readln;
end.
```

5.2)

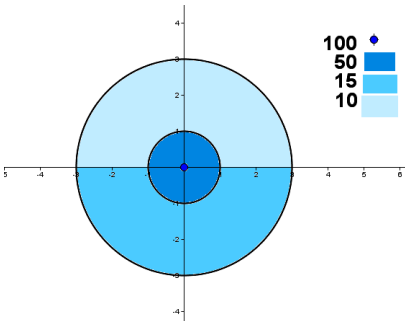
```
Program condi5_2;
var
    y:integer;
begin
    writeln('escriba un número entero');
    readln(y);
    if y>0 then
       if not(odd(y)) then
          writeln(y div 2)
       else
          writeln('El valor ingresado es ',y);
    readln;
end.
```

6) Calcular y mostrar el mínimo de tres valores reales ingresados por teclado.

7) Ingresar 3 caracteres y, si los tres son letras mayúsculas, mostrarlos ordenados alfabéticamente, en caso contrario aclarar que no son datos válidos.

8) Determinar en qué cuadrante se encuentra un punto en el plano teniendo en cuenta la posibilidad de que se encuentre en los ejes.

9) Considerar el juego de tiro al blanco que se muestra a la derecha con los puntajes asignados. Ingresar las coordenadas del tiro realizado por un jugador e informar el puntaje obtenido. Considerar que si un tiro da en una región no sombreada obtiene 0 puntos, y si da en la frontera entre dos regiones considerar el puntaje más alto.



10) Decir cuáles son las salidas del siguiente programa para cada lote de prueba. Justificar las respuestas mediante las pruebas de escritorio.

```
program parcial;
var
    a,b,z,c:integer;
    k:byte;
    t:boolean;
begin
    readln(z,a);
    readln(b);
    t:=true;
```

a) Lote de prueba: 3, 9, 4
b) Lote de prueba: 3, 6, 9
c) Lote de prueba: 12, 9, 7

```
    k:=1;
    if (a div z =0) or (t and (b mod z=0)) then
       c:=z-a
    else
       begin
          c:=a;
          k:=k+1;
          if z<=b then
             case k of
                1: k:=k+1;
                2: c:=b;
                3: c:=a;
             end
          else
             k :=k+2;
       end;
    writeln(c,k);
    readln;
end.
```

11)
a) Escribir un programa que, dadas las componentes real e imaginaria de dos números complejos y uno de los símbolos pertenecientes al conjunto {+, -, *, / }, muestre el resultado de la operación elegida por el usuario.
b) modificar el caso anterior eligiendo la operación con S o s, R o r, M o m y D o d.

12) Para cada uno de los siguientes programas, si es posible, reescribirlos utilizando la sentencia CASE, sin alterar su salida.

```
a)
Program Numeros;
var
    m: integer;
begin
    writeln('Ingrese un nro. entero: ');
    readln(m);
    IF (m=2) or (m=3) THEN
       writeln('Situación 1')
    ELSE
       IF (m>3) and (m<10) THEN
          writeln('Situación 2')
       ELSE
          IF m=10 THEN
             writeln('Situación 3')
          ELSE
             writeln('El valor de m no está en el rango de 2 a 10');
    readln;
end.

b)
Program prg6;
var
    f,x:real;
begin
    readln(x);
    IF x<0 THEN
       f:= sqr(x)
    ELSE
       IF x<5 THEN
          f:= x+1
       ELSE
          IF x=5 THEN
             f:= sqrt(x)
          ELSE
             f:= x-1;
```

```
    writeln('El resultado es ', f:6:2);
    readln;
end.
```

13) Conociendo el IMC, índice de masa corporal (nro. real), informar el estado de la misma de acuerdo con la siguiente tabla y seleccionar la estructura de decisión adecuada para resolver el ejercicio (Justificar esta última elección)

IMC < 18.5	→ 'EXCESIVA DELGADEZ'
18.5 IMC < 25	→ 'PESO IDEAL'
25 IMC < 30	→ 'SOBREPESO'
30 IMC	→ 'OBESIDAD'

14) Escribir un programa que ofrezca al usuario un menú para hacer conversiones de medidas anglosajonas al sistema métrico decimal. El usuario podrá elegir tanto la magnitud como la unidad que desea convertir, y proporcionará al programa el valor a convertir.

Magnitudes	Equivalencias
L- Longitud	De pulgadas a centímetros: 1 pulgada equivale a 2.54 cm De yardas a metros: 1 yarda equivale 0.9144 m De millas a kilómetros: 1 milla equivale a 1.6093 km
S- Superficie	De pulgadas^2 a centímetros^2: 1 pulgada^2 equivale a 6.452 cm^2 De yardas^2 a metros^2: 1 yardas^2 equivale a 0.8361 m^2 De acres a hectáreas: 1 acre equivale a 0.4047 ha
P- Peso	De onzas a gramos: 1 onza equivale a 28.35 g De libras a kilogramos: 1 libra 0.45359 kg De toneladas inglesas a toneladas (métricas): 1 ton inglesa equivale a 1.0160 tm